

Operating Systems Project 4 Memory Management

We have neither given nor received unauthorized assistance on this work

Name of the VM: VladislavUbuntu

Password: ‘ ‘ The password is a single character the spacebar, ascii value of 0x32

Path: /home/vlad/class/project4

Instructions for running:

‘make part1’ builds the first program, and ‘make part2’ makes the second.

Once they are built, load the modules with ‘insmod p1.ko pid_arg=\$MY_PID_ARG’ or

‘insmod p2.ko pid_arg=\$MY_PID_ARG va_string=”my_va_string”’ ‘

Notes:

The first part does print the total size, but only after it is removed. It can be seen by running ‘rmmod Project4_part1’ and then ‘dmesg’

Description of Project:

The project was an exploration of memory mapping for the task_struct. We had to walk thru the maple tree and inspect some of the virtual address tables.

Pseudocode:

I iterate thru the task_struct list to find the one matching the PID argument.

I pass the PID relevent task struct to my helper function, which:

- gets a read_lock

- (in part 1)

- calls the for_each_vma macro

- executes some required math operations on it

- (in part 2)

- iterate through the vma

- check the pages in order and return early if address is not there

Conclusion:

Memory management is a very complicated process. Even at the operating systems level here we are only touching a couple of APIs exposed to us like the `for_each` macros and the `struct`, which we don't really have direct access to since it could change. This project required use of the `read_lock` or the program would crash without it. The 5 layer aspect makes sense because it turns addition into multiplication, because when you stack a level on top of another one, it implies permutations. Though not very intuitive to work with it is an elegant solution to the sparseness of the large virtual address space.

Lessons Learned:

Kernel debugging is hard. Seeing a segment fault was a good thing after hanging the vm because I was dereferencing a null pointer.